EOB Agentic Solution

Interactive Architecture & Technical Documentation

The EOB Agentic Solution is a state-of-the-art **FastAPI** service designed to extract high-accuracy structured data from complex medicinal insurance claims (VSP, EyeMed, etc.).

Self-Healing

Automatic Auditor-Critic loops ensure 100% data integrity without human intervention.

Deep Context

Remembers document history across batches to resolve hierarchical claim dependencies.

Massive Scale

Elastic chunking allows processing of 100+ page PDFs with consistent sub-second page throughput.

01 Ingestion & Synthesis

The API receives the PDF. `FormatLoader` dynamically binds Pydantic schemas, and `PDFProcessor` prepares structural grids using Azure AI.

02 Multimodal Analysis

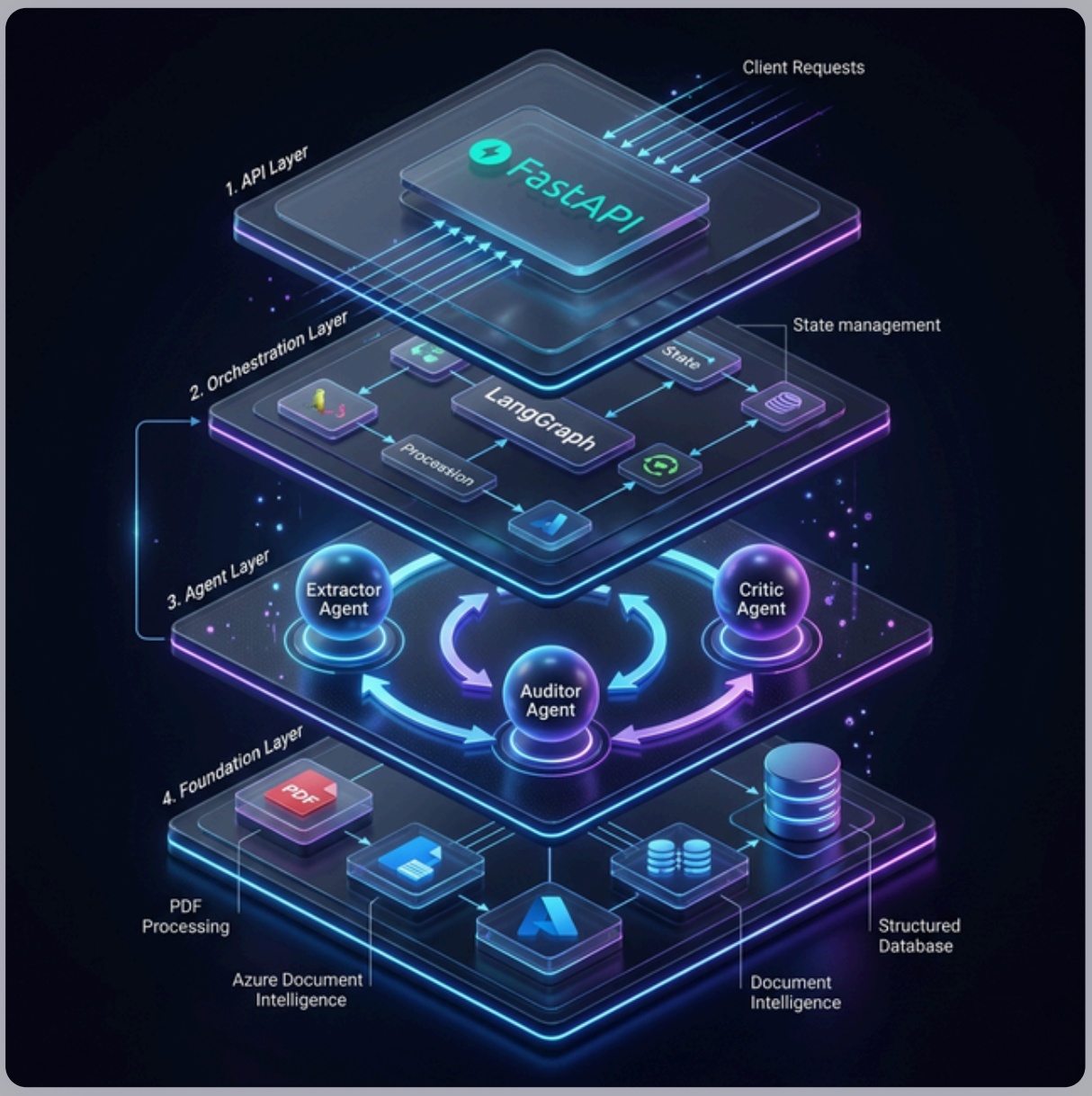
The system fuses OCR text with high-res visual snapshots. This allows the agents to "see" the document layout and table borders perfectly.

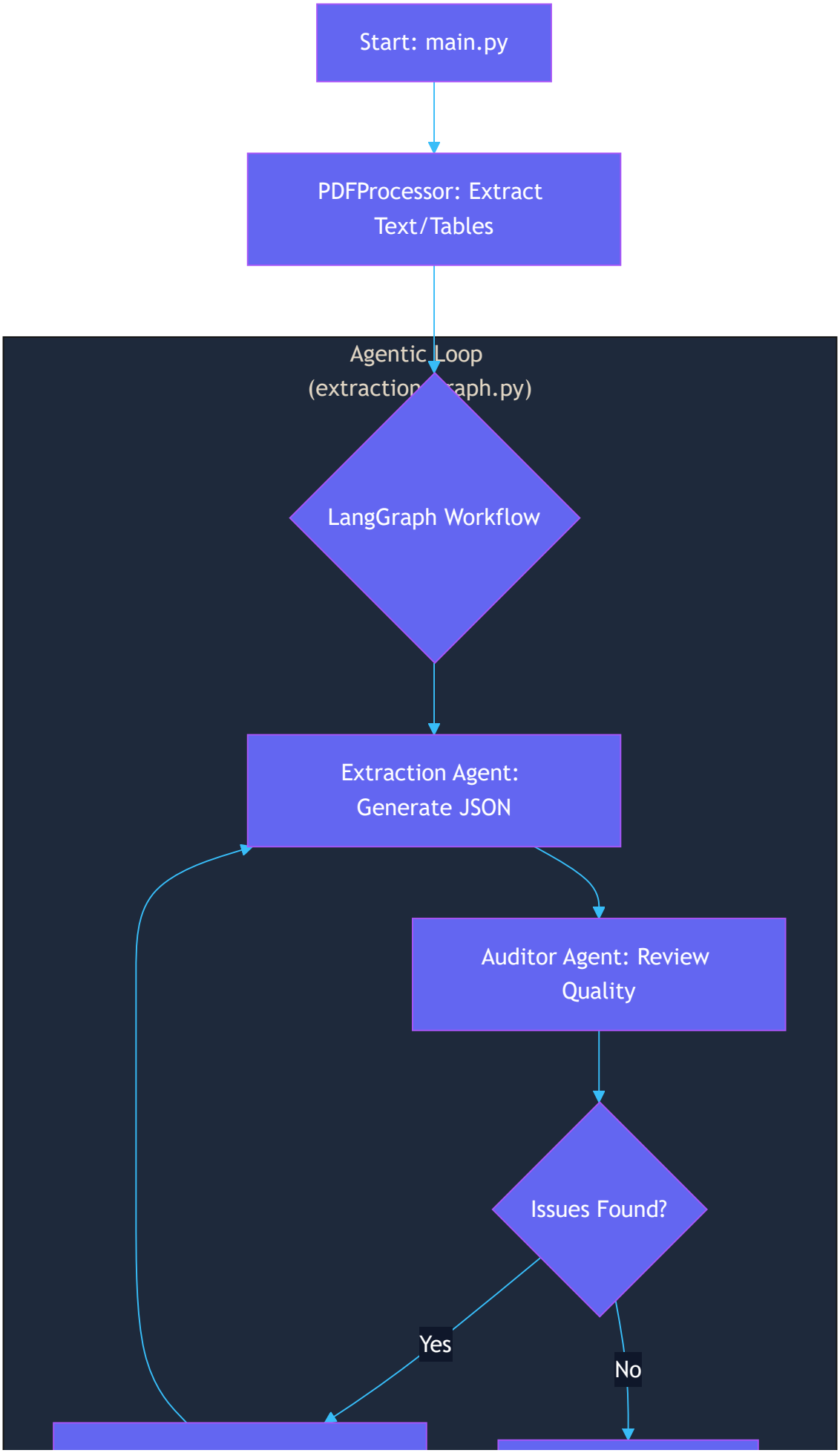
03 The Agentic Crucible

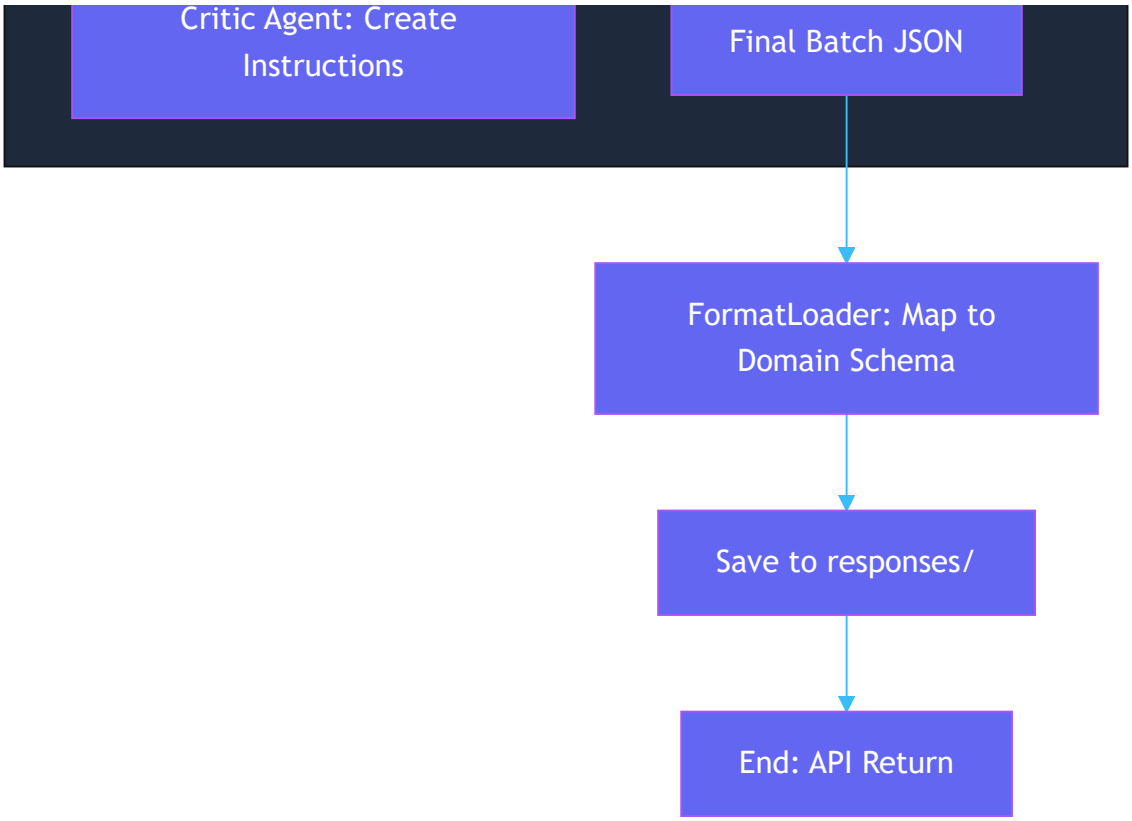
LangGraph orchestrates the feedback loop: Extractor → Auditor → Critic. Missing values are detected and re-extracted automatically.

04 Final Materialization

Results are aggregated, totals are calculated, and the final validated JSON is delivered to the client and saved locally.


Client Requests
1. API Layer
FastAPI
2. Orchestration Layer
State management
State
LangGraph
Procession
3. Agent Layer
Extractor Agent
Critic Agent
Auditor Agent
4. Foundation Layer
PDF
PDF Processing
Azure Document Intelligence
Document Intelligence
Structured Database


Start: main.py
PDFProcessor: Extract Text/Tables
Agentic Loop
(extraction_graph.py)
LangGraph Workflow
Extraction Agent: Generate JSON
Auditor Agent: Review Quality
Issues Found?
Yes
No




```
project_root/
├── main.py                  # API Entrypoint
├── pdf_processor.py         # Azure Document Intelligence
├── agents/                  # The "Brain" (LangGraph + Agents)
├── formats/                 # Provider-specific Schemas
└── responses/               # Extracted Results
```

⚙️ Infrastructure

`main.py`

FastAPI routes and overall orchestration.

`config.py`

Global settings and provider model selection.

🧠 Intelligence

`agents/extraction_graph.py`

The core LangGraph StateGraph logic.

`agents/gemini_agent.py`

The heavy-lifting LLM implementation.

📄 Extraction

`pdf_processor.py`

Azure DI integration for structural parsing.

`formats/`

Collection of Pydantic models for VSP/EyeMed.

🤖

ORCHESTRATOR `extraction_graph`

The "Conductor". Uses **LangGraph** to route data between agents in a cyclic feedback loop, managing the state and retry limits.

EXTRACTOR `gemini_agent.py`

The "Muscle". Multi-modal extraction using **Gemini 1.5 Pro**. Handles continuation context and learning memory.

AUDITOR `auditor_agent.py`

The "Inspector". Acts as a fresh eye, comparing the extracted result against the source PDF to find discrepancies.

CRITIC `critic_agent.py`

The "Coach". Converts Auditor issues into clear, numbered improvement instructions for the next loop.

MANAGER `agent_factory.py`

The "HR". Reads config to hire (instantiate) either Gemini or Claude for the various agent roles.

GENERATOR `format_generator.py`

The "Onboarder". Automatically analyzes new PDF types and generates the Python Pydantic format code.